

IPRE-DICOM: desarrollo y evaluación de una API

Prototipo local para recibir imágenes médicas, describir su calidad técnica y servir modelos de clasificación.

Informe de avance de investigación · Pía Ayala · Septiembre de 2026

Recorrido: necesidad inicial, construcción del sistema, experimento reproducible, resultados y trabajo pendiente.

Problema y pregunta de investigación

Las imágenes médicas llegan en formatos y tamaños diferentes. Antes de analizarlas se necesita abrirlas correctamente, revisar posibles identificadores y conocer qué modelo puede procesarlas.

Pregunta de ingeniería: ¿cómo reunir lectura, indicadores técnicos y predicciones en un servicio reutilizable desde un navegador u otro programa?

Pregunta experimental: ¿puede el servicio cargar un modelo entrenado con un dataset conocido y reproducir sus predicciones?

El avance estudia la integración de software y una primera clasificación experimental.

Objetivo inicial y alcance alcanzado

Necesidad original	Estado actual y evidencia
Formatos, ruido y rotación	Lectura multiformato, métricas y checkpoints previos ejecutados.
Frontal o lateral	Generador de etiquetas IU y entrenador preparados; sin modelo entrenado.
Anonimato	Auditoría y seudonimización de metadata DICOM; sin OCR de píxeles.
Clasificador demostrable	PneumoniaMNIST: normal/neumonía, una época completa y test.
Edad, similitud y clustering	Ideas iniciales; aún no implementadas ni evaluadas.

Qué es la API y qué recibe un usuario

Una API es una interfaz que permite pedir una operación a un programa. En este proyecto, un cliente envía una imagen por HTTP y recibe una respuesta estructurada.

El cliente puede ser el visor web, Swagger (/docs), un script Python o una aplicación externa. Todos pueden llamar al mismo servicio FastAPI.

Ejemplo: enviar una radiografía a POST /predict devuelve dimensiones, indicadores de calidad, revisión de privacidad y predicciones disponibles.

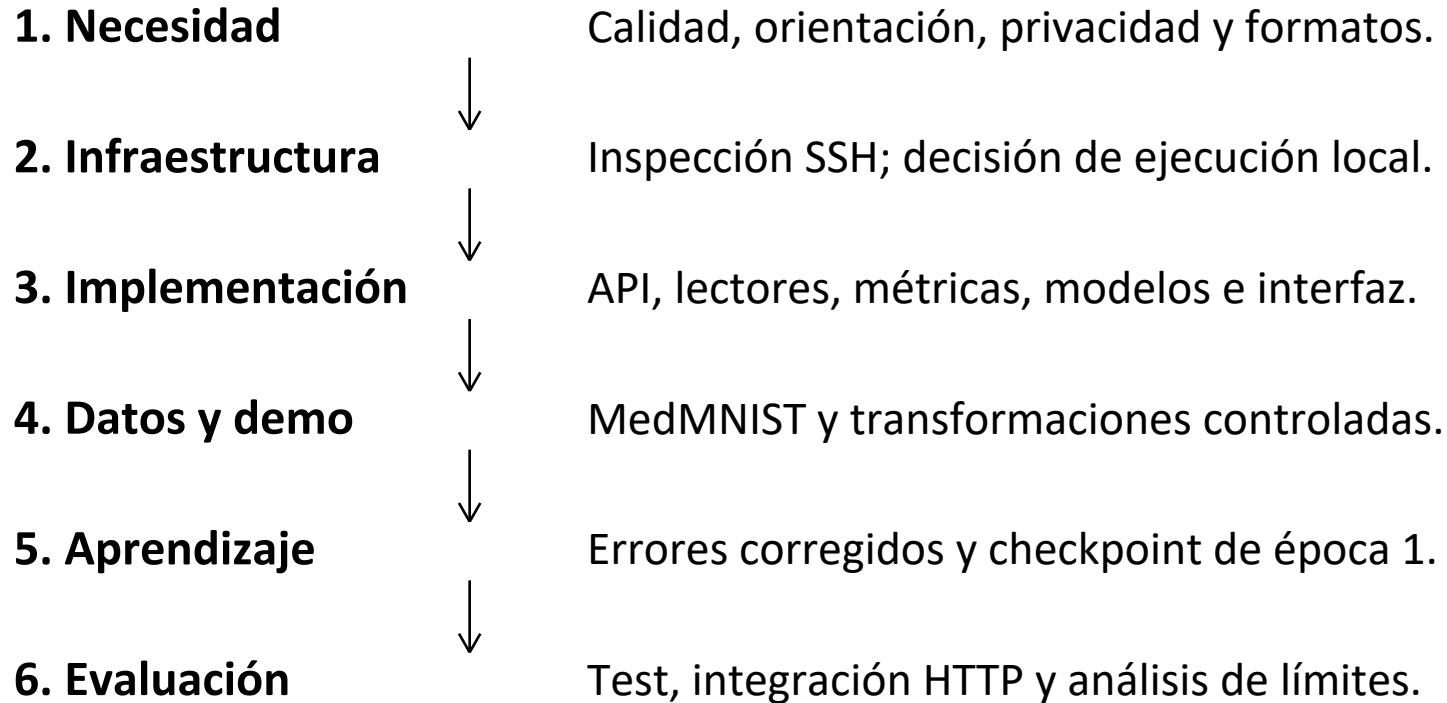
La API usa pesos guardados para predecir. El entrenamiento ocurre por separado, en scripts que producen esos archivos de pesos.

Dos preguntas, dos rutas de análisis

Ruta	Pregunta	Respuesta
POST /predict	¿Qué características técnicas y alertas tiene la imagen?	Métricas, privacidad, ruido/rotación e informe de reglas.
POST /predict/medmnist/{flag}	¿Qué clase del experimento PneumoniaMNIST predice el modelo?	Probabilidad normal y probabilidad neumonía.
GET muestra MedMNIST	¿Qué imagen y etiqueta contiene el dataset?	PNG de la muestra y etiqueta conocida.

Clasificar neumonía demuestra el ciclo de aprendizaje e integración; no valida ruido o rotación.

Recorrido del proyecto: hitos y dependencias



El despliegue en Calfuco sigue pendiente. Los resultados actuales provienen del computador local.

Etapas 1 y 2: necesidad e infraestructura

Etapas	Acción y motivo	Resultado
1. Definir alcance	Separar calidad, orientación, privacidad y clasificación: necesitan etiquetas y evaluaciones distintas.	Conjunto de servicios y tareas identificados.
2. Inspeccionar Calfuco	Revisar GPU, RAM, discos, Python y permisos mediante SSH.	RTX 3090 24 GB, dos 2080 Ti y 125 GB RAM reportados.
Decisión de ejecución	El acceso a workspace y Docker falló; Python del servidor era 3.6.9.	Continuar el prototipo local; conservar plan de despliegue.

Etapas 3 y 4: implementar y preparar datos

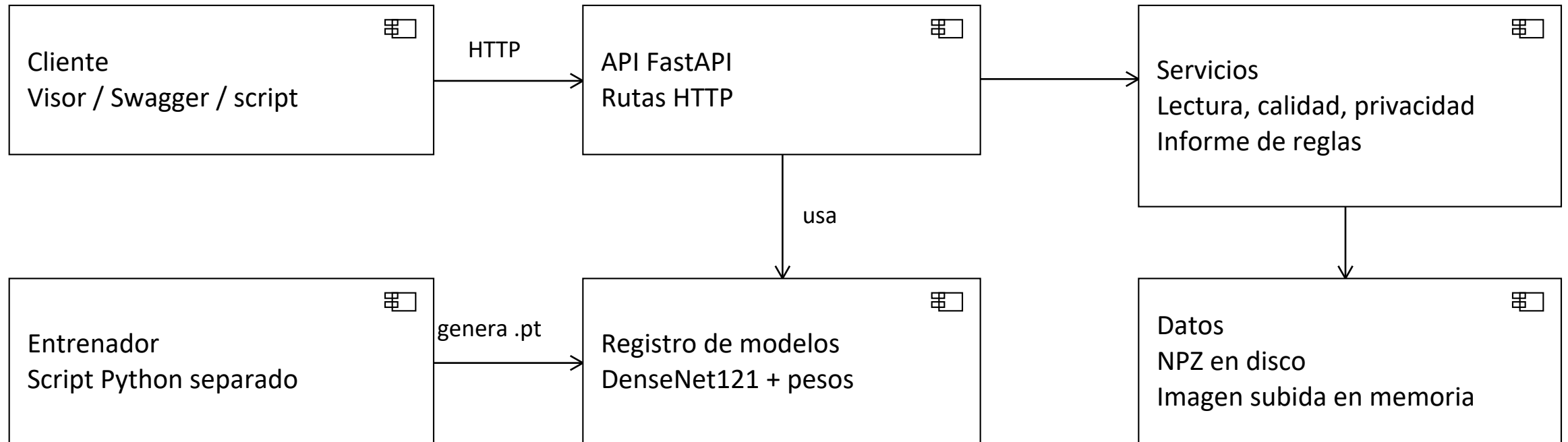
Etapas	Qué se construyó	Por qué fue necesario
3. Servicio modular	FastAPI, visor, lectores, métricas, auditoría e inferencia.	Unificar operaciones tras una interfaz HTTP.
4. Dataset conocido	Descarga de PneumoniaMNIST, catálogo y explorador de muestras.	Contar con imágenes, etiquetas y particiones reproducibles.
Demo controlada	Original del test, copia rotada, copia ruidosa y volumen NPY.	Observar respuestas y fallos en condiciones conocidas.

Etapas 5 y 6: entrenar, corregir y evaluar

Paso observado	Problema o resultado	Decisión aplicada
Primera ejecución	Memoria compartida bloqueada al usar procesos auxiliares.	DataLoader con num_workers=0.
Entrada 28×28	DenseNet121 reducía el mapa espacial hasta tamaño inválido.	Redimensionar a 64×64 ; conservar fuente 28×28 .
Corrida de 5 épocas	Completó época 1; se interrumpió durante época 2.	Evaluar el checkpoint guardado de la época 1.
Integración binaria	La salida positiva aparecía rotulada como normal.	Devolver normal=1-p y neumonía=p; añadir prueba.

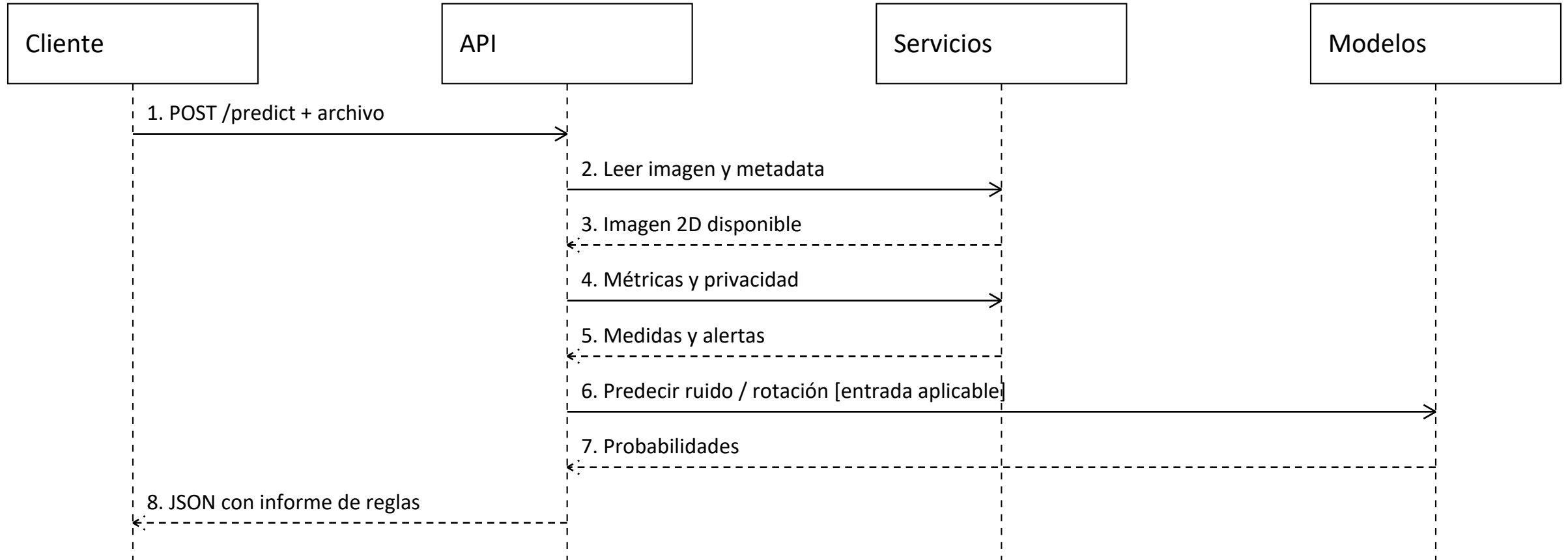
UML de componentes: dónde vive cada responsabilidad

Cada rectángulo es un componente; las flechas indican quién utiliza a quién.



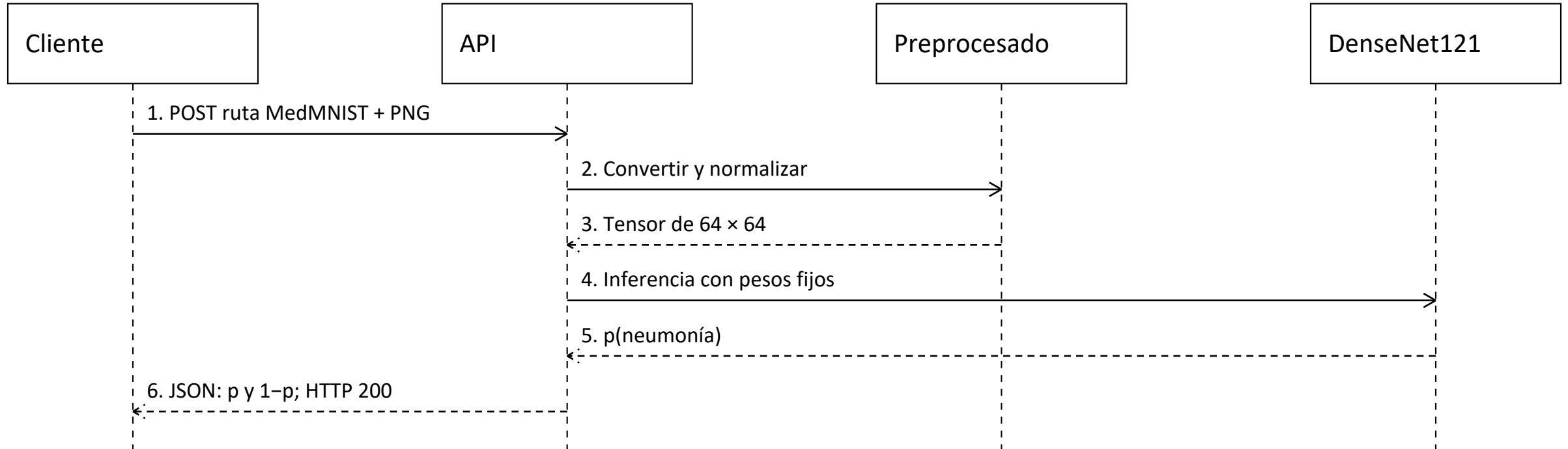
La API corre localmente. Calfuco aún no aloja este sistema. El entrenamiento produce archivos que la API carga después.

UML de secuencia: análisis técnico de una imagen



Lectura: el tiempo avanza de arriba abajo. Flecha continua = solicitud; flecha discontinua = respuesta.

UML de secuencia: usar el modelo de neumonía



El modelo ya está cargado. Esta petición no entrena ni actualiza pesos.

Lectura: el tiempo avanza de arriba abajo. Flecha continua = solicitud; flecha discontinua = respuesta.

Responsabilidad de cada módulo

Módulo	Responsabilidad	Producto que entrega
main.py + visor HTML	Recibir solicitudes y coordinar servicios.	HTTP, interfaz y JSON.
image_io.py + lector DICOM	Interpretar archivo y seleccionar representación 2D.	Imagen normalizada y metadata resumida.
quality.py / privacy.py	Calcular indicadores y buscar campos identificadores.	Medidas y alertas.
medmnist_models.py	Cargar pesos y ejecutar el clasificador del experimento.	Probabilidades por clase.
agent_workflow.py	Convertir hallazgos en recomendaciones mediante reglas.	Informe técnico.
scripts/train*.py	Ajustar y evaluar modelos fuera del servidor HTTP.	Checkpoint y métricas.

Herramientas del servicio y su función

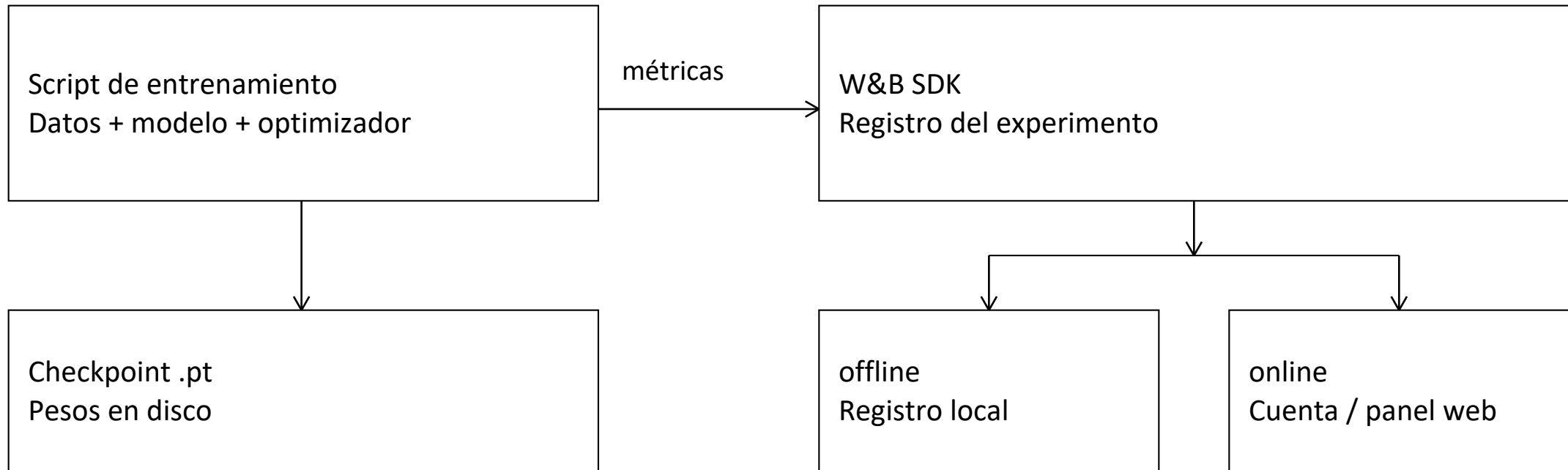
Herramienta	Qué es	Uso concreto
Python	Lenguaje del backend.	Implementa lectura, modelos, endpoints y pruebas.
FastAPI / Uvicorn	Framework web / servidor HTTP.	Define rutas y recibe solicitudes locales.
HTML, CSS, JavaScript	Tecnologías del navegador.	Selección de archivo, envío y visualización de resultados.
Pillow / NumPy	Imágenes y arreglos numéricos.	Escala de grises, tamaños y operaciones sobre píxeles.
pydicom	Biblioteca para DICOM.	Lee metadata y píxeles; exporta copia seudonimizada.

Herramientas de aprendizaje y experimentación

Herramienta	Papel real	Estado
PyTorch	Tensores, gradientes y optimización de pesos.	Entrenamiento e inferencia ejecutados.
MONAI	Biblioteca que aporta DenseNet121 y transformaciones.	Arquitectura usada; MONAI no es un modelo por sí mismo.
MedMNIST	Datasets con muestras, etiquetas y splits.	PneumoniaMNIST instalado y utilizado.
scikit-learn	Funciones para calcular métricas y separar grupos.	Evaluación y pruebas.
W&B	Registro de experimentos, parámetros y artefactos.	Integración y prueba offline; corrida real con W&B desactivado.
LangGraph	Orquestación del informe mediante un grafo.	Un nodo determinista; sin LLM ni aprendizaje.

W&B: dónde se integra en el experimento

W&B actúa como una bitácora: recibe registros del entrenamiento.



Entrenar lo hace PyTorch. W&B registra parámetros, métricas y el archivo del modelo para consultar y comparar experimentos.

W&B: qué programamos en los entrenadores

Momento	Código integrado	Qué queda registrado
Inicio	wandb.init(config=...)	Modelo, semilla, épocas, lote, learning rate y dispositivo.
Fin de cada época	run.log(...)	Loss de train y métricas de validación.
Test de MedMNIST	run.summary[...]	ROC-AUC, F1 macro y accuracy finales.
Cierre	wandb.Artifact + log_artifact	Mejor checkpoint con alias best/latest, seguido de run.finish().

El artefacto se registra al final: una interrupción puede impedir ese paso.

scripts/train.py:82–173; scripts/train_medmnist.py:93–174. Registro habilitado solo si el modo no es disabled.

W&B: implementación, prueba y uso efectivo

1. Implementado

wandb.init, run.log, resumen final y registro del checkpoint en ambos entrenadores.

2. Probado offline

Prueba pequeña con valores de ejemplo. Comprueba el registro, no el rendimiento del clasificador.

3. Corrida real

PneumoniaMNIST se ejecutó con --wandb-mode disabled. Sus métricas quedaron en archivos locales.

Qué se puede mostrar hoy

Código de integración y métricas locales. No existe un panel W&B de la corrida real ni una curva de varias épocas registrada allí.

Siguiente ejecución: elegir offline para registrar localmente u online con una cuenta autenticada. Si se interrumpe, el artefacto final puede no registrarse.

Funcionamiento paso a paso de POST /predict

1. El cliente adjunta el archivo en el campo file. La API rechaza entradas vacías y archivos mayores de 64 MiB.
2. El lector interpreta el formato y obtiene una imagen 2D. Conserva información como dimensiones, modalidad o forma del volumen.
3. Calcula indicadores en la imagen resultante y revisa privacidad. Prepara además una copia de 224×224 para los modelos previos.
4. Para raster o DICOM CR/DX ejecuta los clasificadores disponibles. Otras entradas reciben métricas sin esos clasificadores.
5. El informe reúne hallazgos y recomendaciones. La respuesta JSON regresa al navegador o al programa solicitante.

Lectura y visualización: capacidades y límites

Entrada	Tratamiento actual	Límite relevante
DICOM (.dcm/.dicom)	PixelData, slope/intercept, ventana e inversión MONOCHROME1.	Primer frame; no visor volumétrico.
PNG/JPEG/TIFF/BMP/WebP	Lectura raster y escala de grises.	Vista previa nativa del navegador según formato.
NIfTI (.nii/.nii.gz)	Lectura y selección de cortes centrales hasta 2D.	La interfaz no ofrece navegación 3D.
NumPy (.npy)	Arreglos sin objetos serializados; selección 2D.	Fuera del dominio de clasificadores de tórax.
MedMNIST (.npz)	Archivo de dataset consultado mediante rutas dedicadas.	No se sube como radiografía individual.

Qué significan las medidas de calidad

Indicador	Cálculo implementado	Interpretación permitida
Brillo	Promedio de intensidad entre 0 y 1.	Compara claridad global.
Contraste	Desviación estándar de intensidad.	Describe dispersión de tonos.
Ruido estimado	Mediana del cambio frente a un suavizado local.	Compara variaciones de alta frecuencia.
Nitidez estimada	Promedio de varianzas de diferencias en dos ejes.	Describe variaciones de bordes.

Se calculan con fórmulas y no necesitan entrenamiento. No hay umbral clínico validado.

api/quality.py. Un valor de ruido 0 no prueba ausencia de ruido.

Privacidad e informe de recomendaciones

La auditoría DICOM busca campos identificadores como PatientName. La respuesta informa nombres y conteos de campos, evitando devolver sus valores.

POST /anonymize/dicom genera una copia: vacía identificadores conocidos, elimina campos privados y remapea UIDs con una clave estable.

Los píxeles permanecen intactos. El sistema exige revisión cuando puede haber texto incrustado; no implementa OCR ni borrado automático de ese texto.

LangGraph organiza reglas: ante una probabilidad de ruido/rotación $\geq 0,5$ o una alerta de privacidad, recomienda revisión. No usa un modelo de lenguaje.

Dataset del experimento: PneumoniaMNIST

PneumoniaMNIST contiene radiografías en escala de grises con dos etiquetas: 0=normal y 1=neumonía. Se utilizó el archivo oficial de resolución 28×28 disponible localmente.

Una muestra es una imagen con su etiqueta conocida. El modelo aprende a relacionar patrones de píxeles con esa etiqueta.

La clasificación de neumonía permite verificar un ciclo completo: datos, entrenamiento, checkpoint, evaluación e inferencia HTTP.

Sus etiquetas no describen ruido, rotación, privacidad ni edad. Esas tareas necesitan sus propios datos y criterios de evaluación.

Una muestra: imagen más respuesta conocida



Etiqueta 0: Normal

Test, índice 1 · fuente 28×28



Etiqueta 1: Neumonía

Test, índice 0 · fuente 28×28

Imagen

Los píxeles son la entrada del modelo.

Etiqueta

La respuesta conocida sirve para aprender o evaluar.

Las imágenes se amplían solo para mostrarlas. La etiqueta proviene del dataset, no de una interpretación visual realizada aquí.

Por qué dividimos las imágenes en tres grupos

5.856 imágenes etiquetadas: se asigna una función distinta a cada grupo.



APRENDER

4.708 imágenes · 80,4%

Train: las imágenes y sus etiquetas permiten corregir los pesos.

REVISAR

524 imágenes · 8,9%

Validación: comprobar el modelo al final de la época, sin modificarlo.

EXAMINAR

624 imágenes · 10,7%

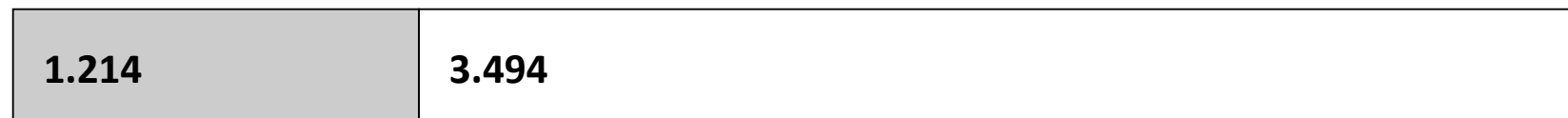
Test: medir el checkpoint elegido con imágenes separadas.

“Ajustar pesos” significa cambiar los números internos del modelo para reducir errores. En validación y test esos números permanecen fijos.

Distribución de clases: qué significa desbalance

Gris = normal; blanco = neumonía. Cada barra representa el 100% de su grupo.

Train



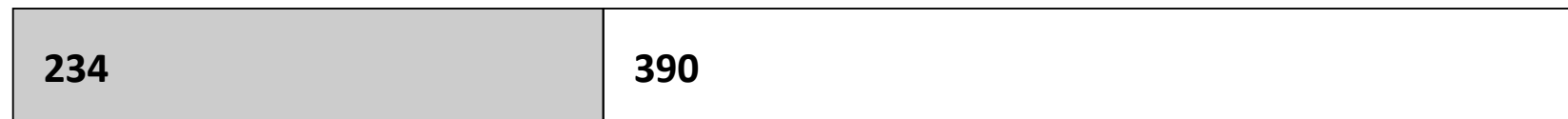
25.8% normal; 74.2% neumonía

Validación



25.8% normal; 74.2% neumonía

Test



37.5% normal; 62.5% neumonía

En test hay más neumonías: responder siempre “neumonía” ya acierta $390/624 = 62,5\%$. Por eso necesitamos una comparación y métricas por clase.

Separación de datos y prevención de fuga

En train se actualizan los pesos. Validación permite decidir qué checkpoint guardar. Test se reserva para medir el modelo ya elegido.

Para PneumoniaMNIST se mantuvieron las particiones oficiales. El archivo utilizado no permite demostrar aquí una auditoría propia de separación por paciente.

El entrenador general admite `source_id`: todas las variantes de una imagen o estudio deben quedar juntas, evitando una original en train y su copia ruidosa en validación.

La muestra test #0 también se usó en la demo. Si se toman decisiones de diseño a partir del test, una etapa futura necesitará otro conjunto externo reservado.

Qué contiene el modelo y qué se guarda

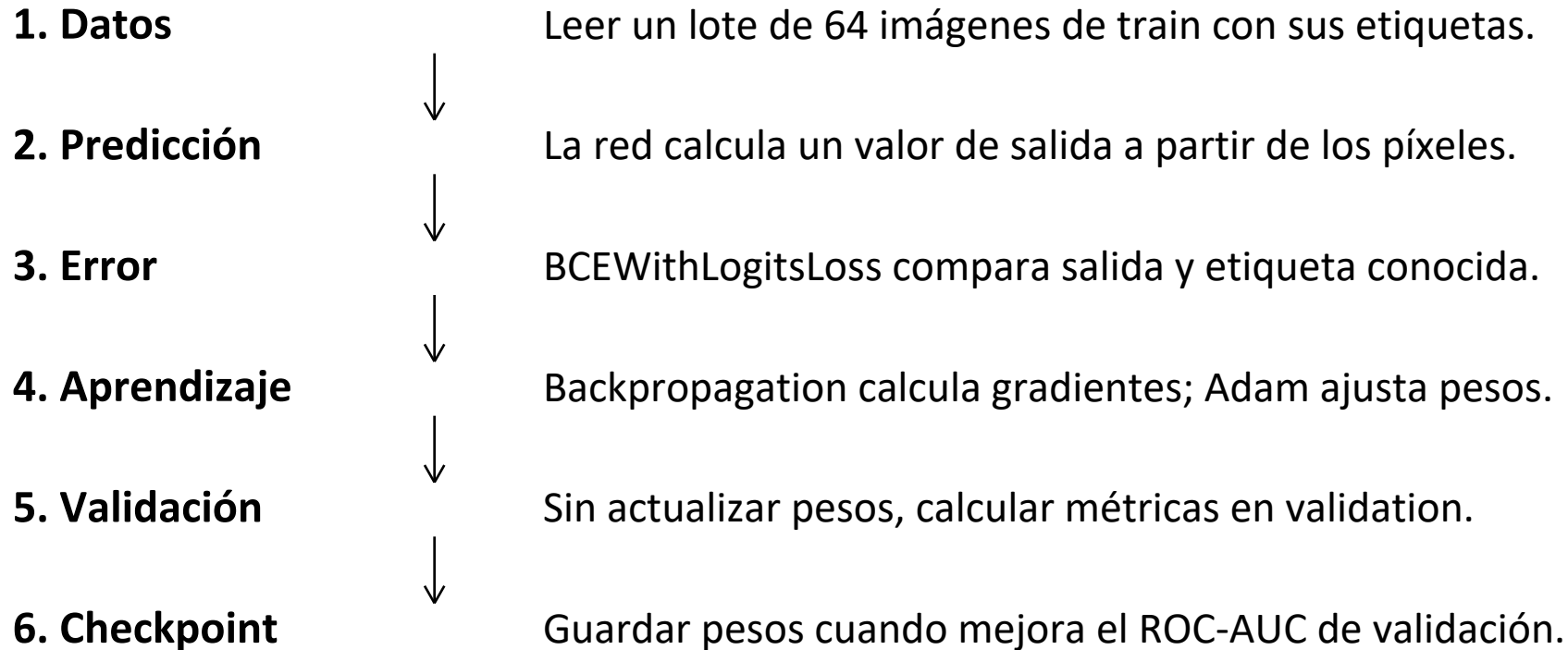
DenseNet121 es una red de capas que transforma píxeles en características numéricas. Sus filtros aprenden patrones de imagen a partir de ejemplos etiquetados.

Las conexiones densas permiten reutilizar características de capas anteriores. La última parte combina esas características para producir la salida de clasificación.

Los pesos son los números que el entrenamiento ajusta. Un checkpoint es un archivo que conserva esos pesos y datos necesarios para reconstruir el modelo.

En este experimento la red produce un número, llamado logit. La función sigmoid lo transforma a un valor entre 0 y 1 asociado con neumonía. El umbral 0,5 convierte ese valor en una clase.

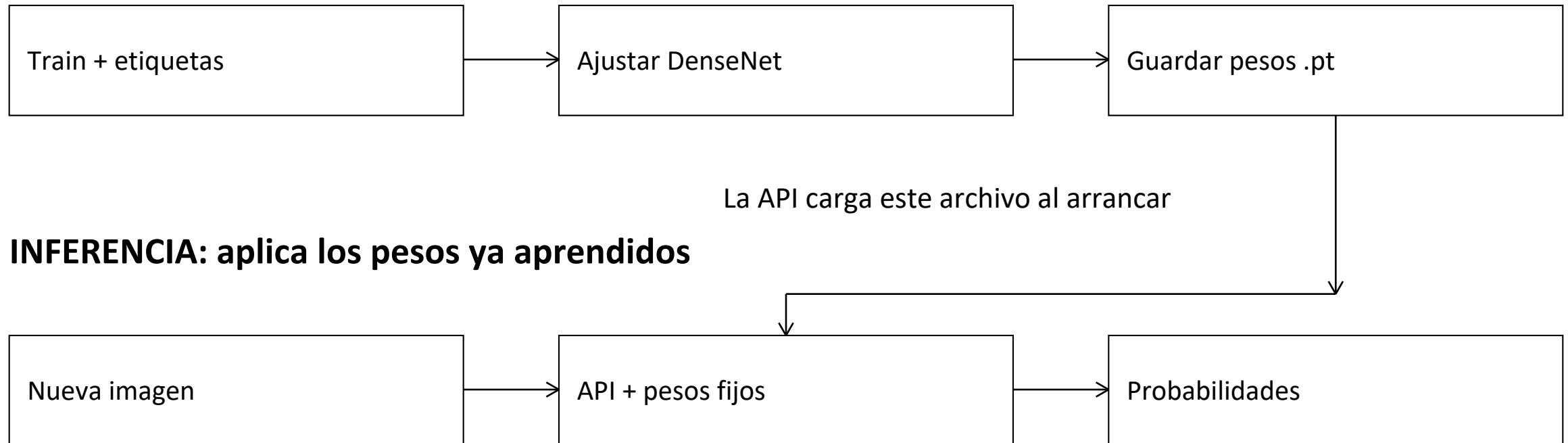
Cómo aprende DenseNet121



Una época recorre todo train. Durante inferencia se usan pesos fijos y no se propaga el error.

Entrenamiento e inferencia: dos momentos diferentes

ENTRENAMIENTO: aprende a partir de imágenes etiquetadas



Enviar una imagen a la API no enseña al modelo. Para aprender de nuevas imágenes hay que ejecutar otro entrenamiento.

Configuración exacta de la corrida inicial

Parámetro	Valor observado o configurado
Modelo	MONAI DenseNet121, 2D, un canal de entrada y una salida; sin pesos preentrenados solicitados.
Entrada	28 × 28 redimensionada a 64 × 64; intensidades normalizadas a aproximadamente [−1, 1].
Optimización	BCEWithLogitsLoss, Adam, learning rate inicial 0,0001, batch de 64.
Plan	5 épocas; CosineAnnealingLR configurado para ese horizonte; semilla 42.
Ejecución real	CPU local, num_workers=0; una época completa y detención durante la segunda.
Modelo evaluado	Checkpoint guardado al terminar la época 1. No se evaluaron los pesos parciales de época 2.

Cómo interpretar las métricas del experimento

Métrica	Significado para este estudio
Accuracy	Fracción de imágenes cuya clase coincide con la etiqueta, usando umbral 0,5.
F1 macro	Media del F1 de normal y neumonía. F1 combina precisión y sensibilidad por clase.
ROC-AUC	Mide cómo ordena positivos frente a negativos al variar el umbral. 0,5 es referencia aleatoria.
Loss	Error usado para ajustar pesos. No es un porcentaje de imágenes incorrectas.
Probabilidad individual	Salida del modelo para una muestra; no equivale a accuracy global ni certeza clínica.

La correlación no se calculó: el experimento evalúa clasificación binaria.

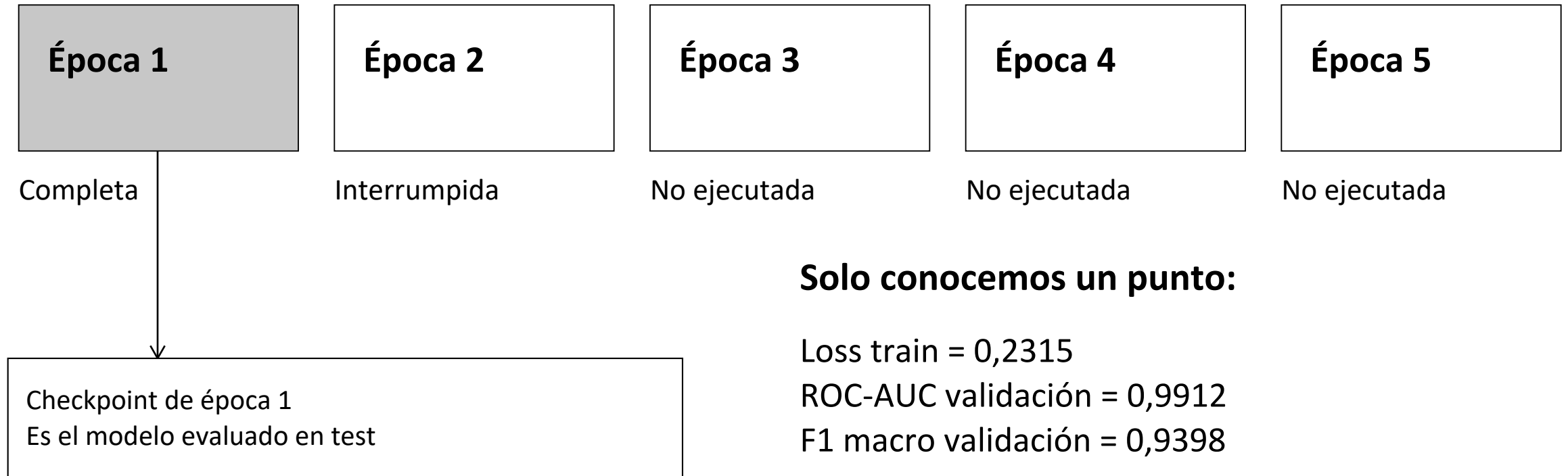
Resultados medidos del checkpoint de época 1

Medición	Validación (524)	Test (624)	Lectura
ROC-AUC	0,9912	0,9478	Descenso de 0,0434 en test.
F1 macro	0,9398	0,8337	Descenso de 0,1061 en test.
Accuracy	No conservada en el registro	85,42%	533 aciertos y 91 errores.
Loss de train	0,2315 al finalizar época 1	No corresponde	Único punto de entrenamiento documentado.

Un punto por época no permite dibujar una curva de aprendizaje ni demostrar convergencia.

Qué se completó realmente durante la corrida

Plan: 5 épocas. Evidencia disponible: una época completa.



Un solo punto no permite evaluar convergencia ni sobreajuste. No se conserva qué fracción de la segunda época alcanzó a ejecutarse.

Traza de ejecución: época 1 completa; Ctrl+C durante época 2; evaluación posterior del checkpoint guardado.

Cómo leer los 624 resultados del test

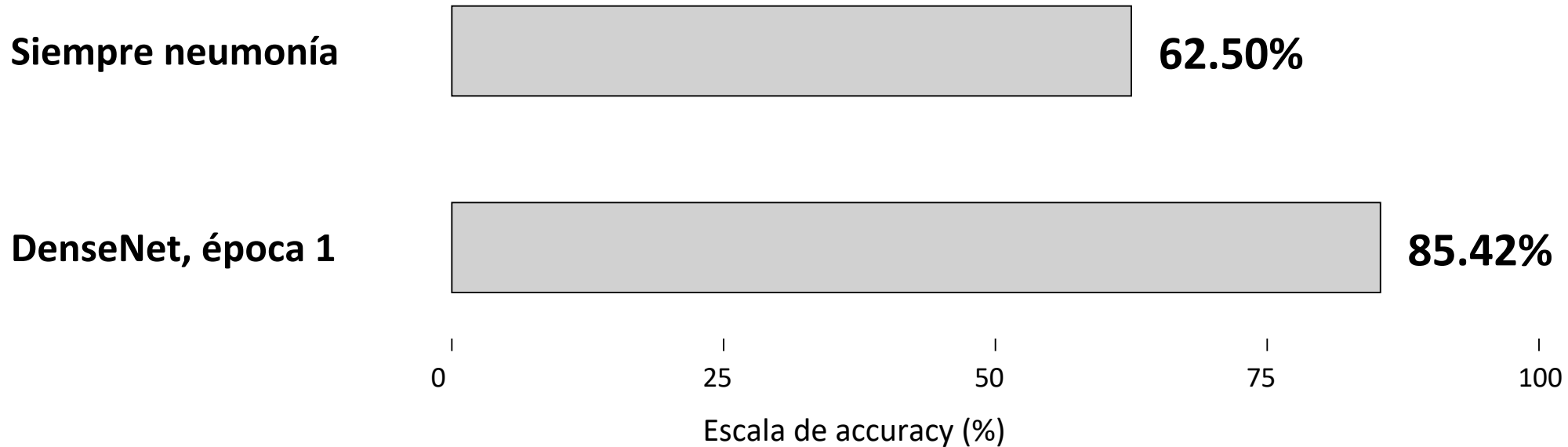
La fila indica la etiqueta conocida; la columna indica la respuesta del modelo.

	Predijo normal	Predijo neumonía
Real: normal	157 Normales reconocidos	77 Falsas alarmas
Real: neumonía	14 Neumonías omitidas	376 Neumonías detectadas

533 aciertos ($157 + 376$) y 91 errores ($77 + 14$). Detecta 96,4% de neumonías, pero reconoce solo 67,1% de normales.

Comparación: ¿el modelo supera una regla trivial?

Accuracy = porcentaje de respuestas correctas sobre las mismas 624 imágenes.



La red mejora la referencia en 22,92 puntos porcentuales. Aún no se comparó contra ResNet ni se repitió con otras semillas.

Comparación de componentes y modelos del proyecto

Componente	Datos / salida	Qué evidencia existe
Métricas de calidad	Píxeles / medidas numéricas.	Demo reproducible; sin ajuste de pesos.
DenseNet ruido y rotación	Checkpoints previos / dos clases por tarea.	Cargan y responden; procedencia de entrenamiento no documentada.
DenseNet PneumoniaMNIST	Splits oficiales / normal-neumonía.	Una época, checkpoint y test medido.
Clasificador de vista	Manifiesto IU / frontal-lateral.	Preparación implementada; sin entrenamiento.
ResNet, ViT, CLIP, DINO, kNN	Alternativas propuestas al inicio.	Sin comparación experimental en este repositorio.

Experimento técnico de ruido y rotación

Caso	Ruido por fórmula	P(ruido)	P(rotación)
Original de MedMNIST	0,0000	0,9465	0,4261
Misma muestra rotada 25°	0,0000	0,9492	0,4278
Misma muestra con ruido añadido	0,0588	0,9943	0,4699
Volumen NPY de demostración	0,0000	No ejecutado	No ejecutado

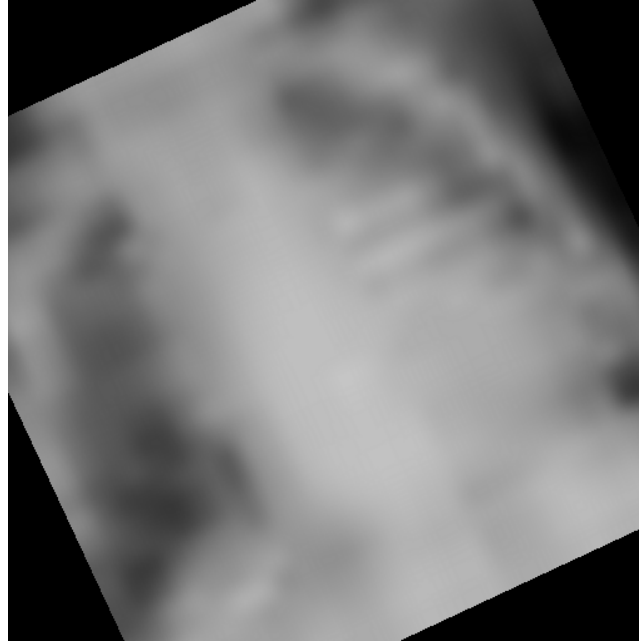
Estas variantes de una sola fuente son una prueba funcional, no un conjunto independiente de evaluación.

Prueba controlada: misma imagen, tres condiciones



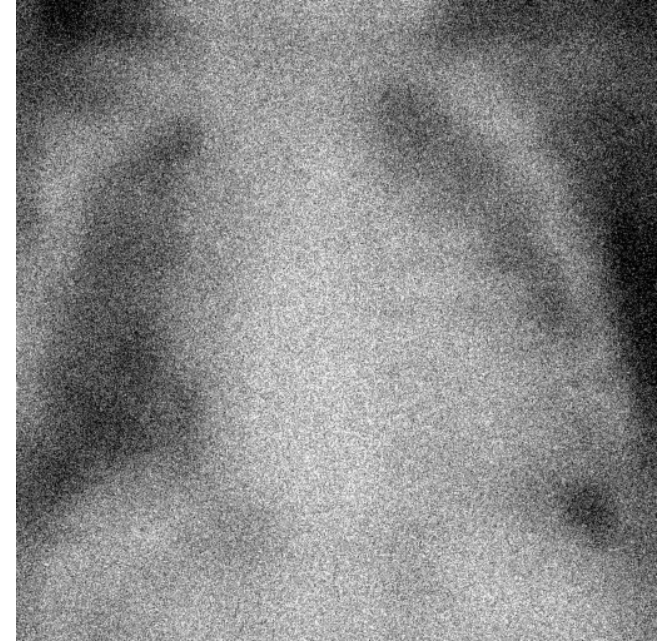
Original

Referencia de la demo



Rotada 25°

Giro artificial conocido



Ruido añadido

Perturbación gaussiana

Se comparan respuestas antes y después de modificar una misma fuente. Este ensayo descubre fallos puntuales; no mide rendimiento poblacional.

Análisis crítico de los hallazgos

La fórmula de ruido aumentó de 0 a 0,0588 con ruido añadido. Esto muestra sensibilidad a esa transformación; no establece un límite de calidad clínica.

El clasificador de ruido asignó 0,9465 a la original. Es una alerta inesperada, pero la imagen no tiene una etiqueta de calidad adjudicada que permita llamarla falso positivo confirmado.

El clasificador de rotación no cruzó 0,5 para la copia girada 25°. Falló en ese ejemplo sintético; no alcanza para estimar sensibilidad global.

DenseNet de neumonía superó la referencia constante. La caída entre validación y test muestra por qué un resultado de validación alto no debe presentarse como rendimiento final.

Del checkpoint a una predicción HTTP

Al iniciar Uvicorn, el registro busca archivos medmnist_*.pt. Lee el dataset, el tamaño de entrada y los pesos para reconstruir DenseNet121.

POST /predict/medmnist/pneumoniamnist recibe un PNG/JPEG, lo convierte a grises, ajusta tamaño, normaliza y calcula una salida.

Sigmoid convierte esa salida en $p(\text{neumonía})$. La API devuelve también $p(\text{normal})=1-p(\text{neumonía})$. No modifica los pesos.

En la prueba test #0, la etiqueta real era 1. El endpoint devolvió HTTP 200, $p(\text{neumonía})=0,98005$ y $p(\text{normal})=0,01995$.

Qué puede mostrar hoy la demostración

Pantalla o acción	Qué muestra	Qué permite comprobar
GET /health	CPU y modelos cargados.	Servidor activo y checkpoint disponible.
Visor /, botón Analizar	Métricas, alertas y modelos previos.	Ruta técnica POST /predict.
Explorador MedMNIST	Muestra test #0, imagen y etiqueta.	Dataset disponible. No ejecuta clasificación.
/docs: ruta de predicción MedMNIST	Subir PNG de demo y ejecutar.	Nueva predicción del checkpoint entrenado.
python scripts/run_demo.py	Tabla y reporte JSON.	Repetición automatizada de la demo.

Guion reproducible de demostración

1. Desde la carpeta ipre-dicom, activar el entorno: `source .venv/bin/activate`
2. Iniciar el servicio: `MEDMNIST_ROOT=data/medmnist python -m uvicorn api.main:app --host 127.0.0.1 --port 8000`
3. Abrir `http://127.0.0.1:8000` y analizar los archivos de `demo_assets`: original, rotada, ruidosa y volumen NPY.
4. Abrir `http://127.0.0.1:8000/docs`. En la ruta MedMNIST usar `flag=pneumoniamnist` y subir `01_original_medmnist.png`.
5. Contrastar una probabilidad individual con la tabla agregada de las 624 imágenes de test. Explicar que miden aspectos distintos.

Verificación y amenazas a la validez

Las 27 pruebas automatizadas verifican contratos HTTP, lectores, reglas, métricas y etiquetas. Pruebas de software aprobadas no miden rendimiento clínico.

El entrenamiento conserva una sola época y una semilla. No hay curvas completas, intervalos de confianza, evaluación externa ni comparación con ResNet.

Hay una diferencia de interpolación por resolver: entrenamiento usa Resize de torchvision; la API usa resize de Pillow sin fijar el mismo filtro. La prueba HTTP verifica integración, no equivalencia numérica completa.

La predicción técnica acepta cualquier raster y la previsualización del navegador es limitada. Falta control semántico de modalidad, visor volumétrico y OCR.

Roadmap siguiente y criterio de término

Hito pendiente	Trabajo concreto	Evidencia esperada
1. Consistencia del pipeline	Unificar interpolación y registrar configuración/épocas de forma incremental.	Prueba de igualdad entre preprocesamientos e historial completo.
2. Experimento comparativo	Entrenar DenseNet y ResNet con protocolo equivalente y varias semillas.	Curvas, métricas por clase y tabla con variabilidad.
3. Calidad y vista	Obtener etiquetas revisadas e incorporar IU frontal/lateral.	Evaluación independiente por tarea.
4. Calfuco	Resolver carpeta, entorno y acceso autorizado a GPU.	Servicio ejecutado allí y prueba HTTP vía túnel SSH.

Conclusiones del avance

Se construyó una API modular que integra lectura de archivos, indicadores técnicos, revisión de metadata y modelos. Un cliente externo puede reutilizar esas operaciones por HTTP.

El ciclo experimental quedó demostrado con PneumoniaMNIST: entrenamiento local, checkpoint de época 1, evaluación sobre 624 muestras y predicción desde la API.

El checkpoint alcanzó accuracy 85,42%, F1 macro 0,8337 y ROC-AUC 0,9478. Superó la referencia constante, aunque la evidencia sigue siendo inicial.

Las pruebas también revelaron límites concretos: clasificadores previos poco documentados, alertas de calidad por revisar, preprocesamiento no idéntico y despliegue remoto pendiente.

Trazabilidad: dónde revisar cada afirmación

Evidencia	Archivo o ubicación
Arquitectura y contratos	api/main.py; api/static/index.html; README.md
Algoritmos del análisis	api/image_io.py; quality.py; privacy.py; anonymize.py; agent_workflow.py
Entrenamiento	scripts/train_medmnist.py; trazas de la sesión
Checkpoint y resultados	models/medmnist_pneumoniamnist_28.pt y archivo *_metrics.json
Demostración	scripts/prepare_demo.py; scripts/run_demo.py; demo_assets/demo_results.json
Pruebas y plan remoto	tests/; DEMO_PRESENTACION.md; CHEATSHEET_CALFUCO.md

Los conteos se obtuvieron del NPZ y los resultados del checkpoint local; no son cifras tomadas de otros estudios.